

Problem Statement: Frequency of Characters

Given a string **S**, write a program to find the frequency of each character present in the string.

Print each character along with the number of times it appears in the string.

Input Format

- A single line containing the string **S**.

Output Format

Print the frequency of each character in the order of their first occurrence.

Format:

character : frequency

Constraints

- $1 \leq \text{Length of } S \leq 1000$
 - String contains only lowercase English letters.
-

Sample Input 1

hello

Sample Output 1

h : 1

e : 1

l : 2

o : 1

Explanation

Character	Frequency
-----------	-----------

h	1
---	---

Character	Frequency
e	1
l	2
o	1

Sample Input 2

programming

Sample Output 2

p : 1
r : 2
o : 1
g : 2
a : 1
m : 2
i : 1
n : 1

Sample Input 3

banana

Sample Output 3

b : 1
a : 3
n : 2

Java Program

```
import java.util.Scanner;
```

```
public class Main {
```

```
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    String str = sc.nextLine();

    boolean[] visited = new boolean[str.length()];

    for (int i = 0; i < str.length(); i++) {

        if (visited[i]) {
            continue;
        }

        int count = 1;

        for (int j = i + 1; j < str.length(); j++) {

            if (str.charAt(i) == str.charAt(j)) {
                count++;
                visited[j] = true;
            }
        }

        System.out.println(str.charAt(i) + " : " + count);
    }
```

```
}  
}
```

Problem Statement: Anagram Check

Given two strings **S1** and **S2**, write a program to check whether they are **anagrams** of each other.

Two strings are said to be **anagrams** if they contain the same characters with the same frequency, but the characters can be arranged in a different order.

If the strings are anagrams, print "**Anagram**". Otherwise, print "**Not Anagram**".

Input Format

- First line contains string **S1**.
- Second line contains string **S2**.

Output Format

Print:

Anagram

or

Not Anagram

Constraints

- $1 \leq \text{Length of } S1, S2 \leq 1000$
- Strings contain only lowercase English letters.

Sample Input 1

listen

silent

Sample Output 1

Anagram

Explanation

Both strings contain the same characters:

listen → e, i, l, n, s, t

silent → e, i, l, n, s, t

Hence, they are anagrams.

Sample Input 2

hello

world

Sample Output 2

Not Anagram

Explanation

The characters and frequencies are different.

Sample Input 3

race

care

Sample Output 3

Anagram

Java Program (Using Sorting)

```
import java.util.Arrays;
```

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```

String str1 = sc.nextLine();
String str2 = sc.nextLine();

if (str1.length() != str2.length()) {
    System.out.println("Not Anagram");
    return;
}

char[] arr1 = str1.toCharArray();
char[] arr2 = str2.toCharArray();

Arrays.sort(arr1);
Arrays.sort(arr2);

if (Arrays.equals(arr1, arr2)) {
    System.out.println("Anagram");
} else {
    System.out.println("Not Anagram");
}
}
}

```

Problem Statement: Longest Palindromic Substring

Given a string **S**, find the **longest substring** that is a palindrome.

A palindrome is a string that reads the same forward and backward.

If there are multiple longest palindromic substrings of the same length, print the one that appears first in the string.

Input Format

- A single line containing the string **S**.

Output Format

Print the longest palindromic substring.

Constraints

- $1 \leq \text{Length of } S \leq 1000$
 - String contains only lowercase English letters.
-

Sample Input 1

babad

Sample Output 1

bab

Explanation

Possible palindromic substrings:

b
a
b
a
d
bab
aba

Both **"bab"** and **"aba"** have length 3.

Since **"bab"** appears first, the output is:

bab

Sample Input 2

cbbd

Sample Output 2

bb

Explanation

Palindromic substrings:

c

b

b

d

bb

Longest palindrome = "**bb**"

Sample Input 3

racecar

Sample Output 3

racecar

Explanation

The entire string is a palindrome.

Sample Input 4

forgeeksskeegfor

Sample Output 4

geeksskeeg

Approach

For every character:

1. Consider it as the center of a palindrome.

2. Expand left and right while characters are equal.
 3. Check both:
 - Odd length palindrome (aba)
 - Even length palindrome (abba)
 4. Store the longest palindrome found.
-

Visualization

For:

babad

Center at:

a

Expand:

bab

Center at:

b

Expand:

aba

Longest length = 3

Output:

bab

Java Program (Optimal Solution)

```
import java.util.Scanner;
```

```
public class Main {
```

```
    static int start = 0;
```

```
    static int maxLength = 1;
```

```

public static void expand(String str, int left, int right) {

    while (left >= 0 &&
           right < str.length() &&
           str.charAt(left) == str.charAt(right)) {

        int currentLength = right - left + 1;

        if (currentLength > maxLength) {
            maxLength = currentLength;
            start = left;
        }

        left--;
        right++;
    }
}

```

```

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    String str = sc.nextLine();

    if (str.length() == 0) {
        System.out.println("");
        return;
    }

    for (int i = 0; i < str.length(); i++) {

        // Odd Length Palindrome
        expand(str, i, i);

        // Even Length Palindrome
        expand(str, i, i + 1);
    }
}

```

```
        System.out.println(
            str.substring(start, start + maxLength)
        );
    }
}
```

1. Favorite Fruits Store

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {

        ArrayList<String> fruits = new ArrayList<>();

        fruits.add("Apple");
        fruits.add("Mango");
        fruits.add("Banana");

        System.out.println(fruits);
    }
}
```

Output

[Apple, Mango, Banana]

2. First Student Name Print

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {

        ArrayList<String> students = new ArrayList<>();

        students.add("Saran");
        students.add("Kumar");
```

```
        students.add("Priya");

        System.out.println(students.get(0));
    }
}
```

Output

Saran

3. Add Two Numbers and Print

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {

        ArrayList<Integer> numbers = new ArrayList<>();

        numbers.add(10);
        numbers.add(20);

        System.out.println(numbers);
    }
}
```

Output

[10, 20]

4. Find Total Students

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {

        ArrayList<String> students = new ArrayList<>();

        students.add("A");
```

```
        students.add("B");
        students.add("C");
        students.add("D");

        System.out.println(students.size());
    }
}
```

Output

4

5. Change a Name

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {

        ArrayList<String> names = new ArrayList<>();

        names.add("Saran");
        names.add("Kumar");

        names.set(1, "Ravi");

        System.out.println(names);
    }
}
```

Output

[Saran, Ravi]

6. Remove a Fruit

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
```

```
ArrayList<String> fruits = new ArrayList<>();

fruits.add("Apple");
fruits.add("Banana");
fruits.add("Mango");

fruits.remove("Banana");

System.out.println(fruits);
}
}
```

Output

[Apple, Mango]

7. Print All Colors

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {

        ArrayList<String> colors = new ArrayList<>();

        colors.add("Red");
        colors.add("Blue");
        colors.add("Green");

        for(int i = 0; i < colors.size(); i++) {
            System.out.println(colors.get(i));
        }
    }
}
```

Output

Red
Blue
Green

8. Check a Name Exists

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {

        ArrayList<String> names = new ArrayList<>();

        names.add("Saran");
        names.add("Kumar");

        System.out.println(names.contains("Saran"));
    }
}
```

Output

true

9. Last Element Print

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {

        ArrayList<String> fruits = new ArrayList<>();

        fruits.add("Apple");
        fruits.add("Mango");
        fruits.add("Banana");

        System.out.println(fruits.get(fruits.size()-1));
    }
}
```

```
}  
}
```

Output

Banana

10. Classroom Attendance Example

```
import java.util.ArrayList;  
  
public class Main {  
    public static void main(String[] args) {  
  
        ArrayList<String> attendance = new ArrayList<>();  
  
        attendance.add("Saran");  
        attendance.add("Priya");  
        attendance.add("Ravi");  
  
        System.out.println("Attendance: " + attendance);  
    }  
}
```

Output

Attendance: [Saran, Priya, Ravi]

11. Add 5 Mobile Brands

Question:

5 mobile brands add செய்து print செய்யவும்.

```
ArrayList<String> mobiles = new ArrayList<>();
```

```
mobiles.add("Samsung");  
mobiles.add("Vivo");  
mobiles.add("Oppo");  
mobiles.add("Realme");  
mobiles.add("iPhone");
```



```
System.out.println(mobiles);
```

12. Print Second Subject

```
ArrayList<String> subjects = new ArrayList<>();
```

```
subjects.add("Maths");  
subjects.add("Science");  
subjects.add("English");
```

```
System.out.println(subjects.get(1));
```

Output

Science

13. Store Roll Numbers

```
ArrayList<Integer> rollNos = new ArrayList<>();
```

```
rollNos.add(101);  
rollNos.add(102);  
rollNos.add(103);
```

```
System.out.println(rollNos);
```

14. Find Number of Fruits

Question:

How many fruits are there?

```
ArrayList<String> fruits = new ArrayList<>();
```

```
fruits.add("Apple");  
fruits.add("Mango");  
fruits.add("Banana");
```

```
System.out.println(fruits.size());
```

Output

3

15. Replace a Color

```
ArrayList<String> colors = new ArrayList<>();
```

```
colors.add("Red");  
colors.add("Blue");  
colors.add("Green");
```

```
colors.set(1, "Yellow");
```

```
System.out.println(colors);
```
